

MATH 629 Homework 2

Mansur Alimbayev

Theoretical Problem

Problem 4.

Problem Setup.

We need to show that for a compact set $\Omega \subset \mathbb{R}^d$ with nonempty interior, and $\mathcal{C}(\Omega)$ the space of continuous functions on Ω with the uniform norm $\|\cdot\|_\infty$.

If the activation $\sigma: \mathbb{R} \rightarrow \mathbb{R}$ is continuous and integers $W, L \geq 1$, we let $\mathcal{Y}_\sigma^{W,L}(\mathbb{R}^d)$ be the set of functions realized on Ω by depth- L , width- W fully connected feedforward neural networks with activation σ in the hidden layers and an affine output layer.

We need to show that if σ is not a polynomial, then for every fixed $L \geq 1$,

$$\overline{\bigcup_{W \geq 1} \mathcal{Y}_\sigma^{W,L}(\mathbb{R}^d)} = \mathcal{C}(\Omega),$$

where the bar denotes closure in $\|\cdot\|_\infty$. In other words, depth- L networks of arbitrary width can approximate any continuous function on Ω .

i. Step 0: Using the Universal Approximation Theorem (UAT).

For $L=1$, the result follows from the Universal Approximation Theorem (UAT) (Leshno et al. 1993): which states that - if σ is continuous and not a polynomial, then single-hidden-layer networks (depth 1, one hidden layer plus an affine output) with arbitrary width are dense in $\mathcal{C}(\Omega)$.

Therefore,

$$\overline{\bigcup_{W \geq 1} \mathcal{Y}_\sigma^{W,1}(\mathbb{R}^d)} = \mathcal{C}(\Omega). \quad (\text{UAT})$$

This provides the base case for extending to deeper networks. Our goal is to extend this universality to any fixed depth $L \geq 1$.

ii. Step 1: Approximating the identity with σ -networks.

Since σ is non-polynomial and continuous by UAT, we can approximate identity map. For any compact set $K \subset \mathbb{R}^W$ and any $\eta > 0$, there exists a one-hidden-layer σ -network $h: K \rightarrow \mathbb{R}^W$ such that:

$$\sup_{z \in K} \|h(z) - z\|_2 < \eta \text{ for all } z \in K$$

This is achieved by approximating each coordinate function $z \mapsto z_i$ within $\eta/\sqrt{W}$, and then combining them with a linear output layer. Since K is compact, all such maps are bounded and Lipschitz, which allows uniform error control.

iii. *Step 2: Embedding shallow networks into depth- L networks.*

Let's take $f \in C(\Omega)$ and $\varepsilon > 0$. By UAT, there exists a one-hidden-layer network :

$$g(x) = a_0 + a^T \sigma(Bx + b),$$

with $B \in R^{W \times d}$, $b \in R^W$, $a \in R^W$, $a_0 \in R$, such that :

$$\sup_{x \in \Omega} |f(x) - g(x)| < \varepsilon/2. \quad (2)$$

Then we can define the compact set:

$$K = \{\sigma(Bx + b) : x \in \Omega\} \subset R^W$$

Now, for any $\delta > 0$, by Step 1 we can construct $(L - 1)$ one-hidden-layer σ -networks

$$J_\delta^{(1)}, \dots, J_\delta^{(L-1)} : K \rightarrow R^W$$

such that:

$$\sup_{z \in K} \|J_\delta^{(m)}(z) - z\|_2 < \delta, \quad m = 1, \dots, L - 1. \quad (3)$$

Then let,

$$\Phi_{L-1} = J_\delta^{(L-1)} \circ \dots \circ J_\delta^{(1)}.$$

iv. *Step 3: Error propagation through compositions.*

Since each $J_\delta^{(m)}$ is δ -close to the identity and the domain is compact, we can get:

$$\sup_{z \in K} \|\Phi_{L-1}(z) - z\|_2 \leq C(L) \delta, \quad (4)$$

for some constant $C(L)$ (a crude bound is $C(L) = 2(L - 1)$).

Therefore,

$$\sup_{x \in \Omega} |a^T \Phi_{L-1}(\sigma(Bx + b)) - a^T \sigma(Bx + b)| \leq \|a\|_2 C(L) \delta. \quad (5)$$

Choosing $\delta > 0$ such that $\|a\|_2 C(L) \delta < \varepsilon/2$, and combining with (2), we obtain:

$$\sup_{x \in \Omega} |G_L(x) - g(x)| < \varepsilon / 2, \quad (6)$$

where, $G_L(x) = a_0 + a^T \Phi_{L-1}(\sigma(Bx + b))$.

Combining this with $|f - g|_\infty < \varepsilon/2$, we conclude:

$$|f - G_L|_\infty < \varepsilon$$

Therefore, G_L is realized by a depth- L σ -network:

- The first hidden layer computes $\sigma(Bx + b)$,
- The next $(L - 1)$ hidden layers implement the approximate-identity composition Φ_{L-1} ,
- Finally, the affine output layer computes $a_0 + a^T(\cdot)$.

v. Step 4: Necessity of Non-Polynomiality.

If σ is a polynomial of degree m , then every network output would be a polynomial in the inputs of degree at most m^L . This is a finite-dimensional space and cannot be dense in $\mathcal{C}(\Omega)$ unless Ω is finite. Hence, the non-polynomial assumption is essential.

vi. Conclusion.

Starting from the single-hidden-layer UAT and inserting $(L - 1)$ approximate identity layers, we embed shallow approximants into depth- L architectures. Uniform error bounds under composition show that for any fixed $L \geq 1$, and arbitrary width, σ -networks are dense in $\mathcal{C}(\Omega)$.

Therefore:

$$\overline{\cup_{W \geq 1} \mathcal{Y}_\sigma^{\mathcal{W}, \mathcal{L}}(R^d)} = \mathcal{C}(\Omega), \quad \forall L \geq 1.$$

Computational Problem

Continuation from HW1

1. Problem setup

We approximate two functions on $[0,1]^2$:

- $f_1(x, y) = \sin(2\pi x) \sin(2\pi y)$ (regression).
- $f_2(x, y) = \mathbb{1}[x > y]$ (binary classification with the diagonal $x=y$ labeled 0).

2. Data

We draw $(x, y) \sim \text{Unif}([0,1]^2)$ and split 90/10 into train/test sets. This yields a balanced classification set for f_2 since $P(x>y) = 0.5$ under the uniform distribution.

3. Models

Multilayer perceptrons (MLPs) with width $H \in \{32, 64\}$ (primary runs used $H = 64$ to match the summary):

- Shallow: $2 \rightarrow H \rightarrow 1$.
- Deep: $2 \rightarrow H \rightarrow H \rightarrow 1$.

For each depth we test Tanh and ReLU activations on both f_1 and f_2 .

4. Training

We compare two training methods:

- Vanilla SGD: (learning rate $= 5 \times 10^{-2}$, momentum $= 0$)
- SGD + momentum (learning rate $= 5 \times 10^{-2}$, momentum $= 0.9$).

All models are trained for 120 epochs with batch size 128.

Losses:

- 120 epochs, batch size = 128.
- MSE for f_1 ;
- BCEWithLogitsLoss for f_2 (logit output).

This loss function combines a sigmoid activation and binary cross-entropy loss into a single, numerically stable operation. It is the standard for binary classification as it prevents the intermediate calculation of probabilities, which can lead to numerical instability during training.

Metrics:

- Test MSE for f_1 ;
- Test Accuracy and AUROC for f_2 .

Test Accuracy and Area Under the Receiver Operating Characteristic Curve (AUROC). While accuracy measures the fraction of correct predictions, AUROC evaluates the model's ability to rank positive examples higher than negative ones across all possible classification thresholds. An AUROC of 1.0 indicates perfect separability, meaning the model's confidence is perfectly aligned with the true classes.

5. Visualization:

For each task, we render:

1. The *ground-truth* field on a 200×200 grid, this is a plot of the true f_1 or f_2 function. It serves as the essential "answer key," providing a visual benchmark against which to compare the model's predictions.
2. The *model's prediction* (real-valued surface for f_1 ; probability map for f_2). These visuals complement the scalar metrics: the f_1 plots show whether the four-lobe sinusoid is captured; the f_2 plots reveal how sharply probabilities change across the diagonal $x = y$.

We also, are comparing the learning rates for each of the training algorithms.

6. Results:

- f_1 (regression):
 - Deep + ReLU + SGD+momentum: best performance, $\text{MSE} \approx 3.2 \times 10^{-4}$.
 - Deep + ReLU + vanilla SGD: worse, $\text{MSE} \approx 7.6 \times 10^{-4}$.
 - Deep + Tanh + SGD+momentum: $\text{MSE} \approx 6.1 \times 10^{-4}$.
 - Shallow models: significantly worse, especially Shallow + Tanh + SGD $\approx 9.7 \times 10^{-3}$.

Heatmaps confirm accurate recovery of the four-lobe structure for the deep models.

- f_2 (classification):
 - Both optimizers achieve $\text{ACC} \approx 0.991\text{--}0.999$ and $\text{AUROC} \approx 1.0$.
 - The task is linearly separable, so depth/activation/optimizer differences are negligible.
 - Momentum accelerates convergence: AUROC saturates in ~ 20 epochs with momentum vs. ~ 60 with vanilla SGD.

task	depth	activation	optimizer	MSE
f1	deep	relu	sgdm	0.000324
f1	deep	tanh	sgdm	0.000610
f1	shallow	relu	sgdm	0.001886
f1	shallow	tanh	sgdm	0.009683
f1	deep	relu	sgd	0.027433
f1	deep	tanh	sgd	0.046798
f1	shallow	relu	sgd	0.070915
f1	shallow	tanh	sgd	0.153286

Figure 1. The Summary table for Function 1.

task	depth	activation	optimizer	ACC	AUROC
f2	shallow	tanh	sgdm	0.99875	1.000000
f2	deep	tanh	sgd	0.99875	0.999994
f2	shallow	relu	sgd	0.99875	0.999987
f2	shallow	relu	sgdm	0.99750	0.999987
f2	deep	relu	sgdm	0.99625	0.999994
f2	deep	relu	sgd	0.99625	0.999987
f2	shallow	tanh	sgd	0.99375	1.000000
f2	deep	tanh	sgdm	0.99125	1.000000

Figure 2. Summary table for Function 2.

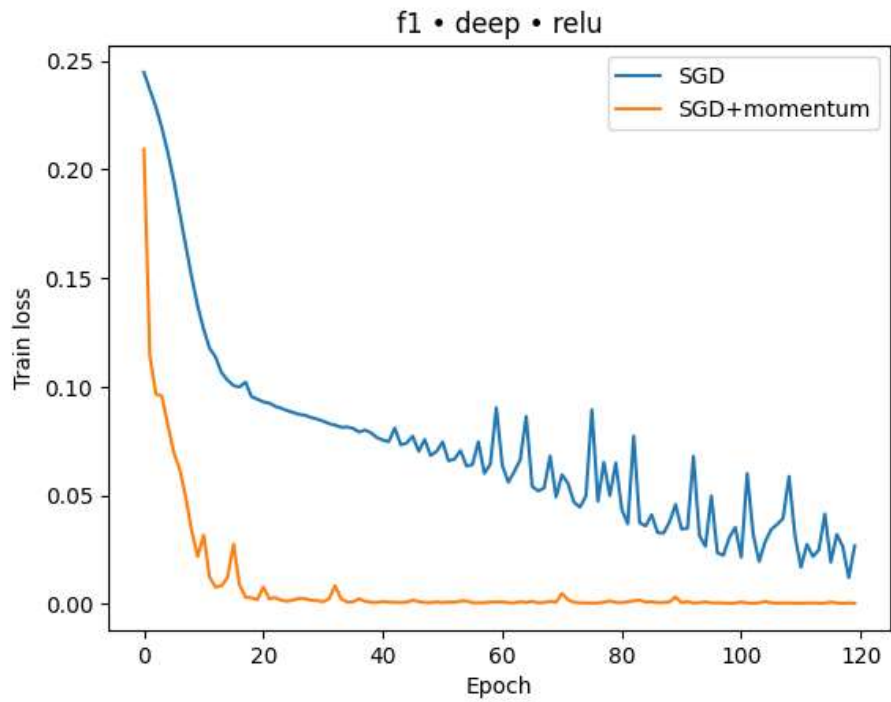


Figure 3. Comparison of Train Loss SGD vs SGD+ momentum

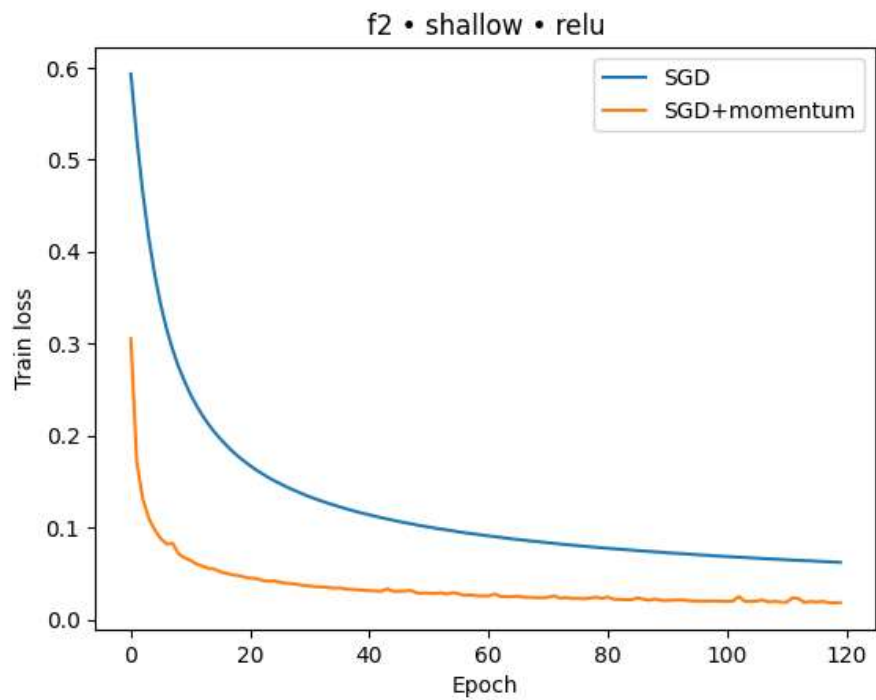


Figure 4. Comparison of Train Loss SGD vs SGD+ momentum

7. Discussion

a) Activation vs. target regularity.

- f_1 is smooth and periodic. Tanh + depth matches this structure well, but ReLU + depth also approximates strongly.
- f_2 is linearly separable, so even shallow networks would be sufficient.

b) Effect of momentum.

- Without momentum, SGD converges slowly and can stagnate with oscillations.
- With momentum (0.9), convergence is much smoother and faster. Final MSE is consistently lower for f_1 .
- For f_2 , both optimizers reach the same near-perfect solution, but momentum requires fewer epochs.

c) Comparison with Adam (Main difference from HW1).

- Adam handled shallow Tanh better at default hyperparameters.
- Here, vanilla SGD performed worse without careful tuning, while momentum restored efficiency close to Adam.

d) Definition of f_2 .

We use:

$$f_2(x, y) = \begin{cases} 1 & x > y \\ 0 & x \leq y \end{cases}$$

So, points on the diagonal are assigned to class 0. Under continuous sampling $P(x = y) = 0$, preserving class balance at ≈ 0.5 ; on a mesh, this convention anchors the decision boundary.

8. Appendix: Code and Figures.

Code snippet: Link to Google Colab: [Link](#)

```
#Math 629
#Homework 2
#Mansur Alimbayev

import numpy as np, torch, torch.nn as nn, torch.nn.functional as F
from torch.utils.data import TensorDataset, DataLoader
from sklearn.metrics import accuracy_score, roc_auc_score
import matplotlib.pyplot as plt
from math import pi
import pandas as pd

# ----- Config -----
SEED = 7
N_SAMPLES = 4000
TEST_FRAC = 0.2
BATCH = 128
EPOCHS = 120
HIDDEN = 64
ACTS = ["tanh", "relu"]
DEPTHS = ["shallow", "deep"]
OPTS = ["sgd", "sgdm"] # vanilla vs momentum

torch.manual_seed(SEED); np.random.seed(SEED)

# ----- Data -----
def make_data(n=N_SAMPLES):
    X = np.random.rand(n,2).astype(np.float32)
    f1 = np.sin(2*pi*X[:,0]) * np.sin(2*pi*X[:,1]) # regression
    f2 = (X[:,0] > X[:,1]).astype(np.float32) # 1 if x>y
    else 0 (x<=y → 0)
    return X, f1[:,None].astype(np.float32), f2[:,None].astype(np.float32)

def split(X, y, test_frac=TEST_FRAC, seed=0):
    rng = np.random.default_rng(seed)
    idx = rng.permutation(len(X))
    k = int(len(X)*(1-test_frac))
    tr, te = idx[:k], idx[k:]
    return X[tr], X[te], y[tr], y[te]

X, y1, y2 = make_data()
# use fixed splits for fairness
X1tr,X1te,y1tr,y1te = split(X, y1, seed=1)
X2tr,X2te,y2tr,y2te = split(X, y2, seed=1)

# ----- Models -----
class MLP(nn.Module):
```

```

def __init__(self, depth="shallow", hidden=HIDDEN, act="relu"):
    super().__init__()
    A = {"relu": nn.ReLU(), "tanh": nn.Tanh()}[act]
    if depth == "shallow":
        self.net = nn.Sequential(nn.Linear(2, hidden), A, nn.Linear(hidden, 1))
    else:
        self.net = nn.Sequential(
            nn.Linear(2, hidden), A,
            nn.Linear(hidden, hidden), A,
            nn.Linear(hidden, 1)
        )
    def forward(self, x): return self.net(x)

def make_opt(params, name):
    if name == "sgd":
        return torch.optim.SGD(params, lr=5e-2, momentum=0.0)
    elif name == "sgdm":
        return torch.optim.SGD(params, lr=5e-2, momentum=0.9)
    else:
        raise ValueError(f"Unknown optimizer: {name}")

# ----- Train/Eval -----
def train_reg(depth, act, opt_name, return_curve=False):
    model = MLP(depth, HIDDEN, act)
    opt = make_opt(model.parameters(), opt_name)
    crit = nn.MSELoss()
    dl = DataLoader(TensorDataset(torch.from_numpy(X1tr), torch.from_numpy(y1tr)),
                    batch_size=BATCH, shuffle=True)

    curve = []
    for _ in range(EPOCHS):
        for xb, yb in dl:
            opt.zero_grad(); loss = crit(model(xb), yb); loss.backward(); opt.step()
        if return_curve:
            with torch.no_grad():
                curve.append(crit(model(torch.from_numpy(X1tr)),
                                torch.from_numpy(y1tr)).item())
        with torch.no_grad():
            mse = F.mse_loss(model(torch.from_numpy(X1te)), torch.from_numpy(y1te)).item()
    return (model, {"MSE": mse, "curve": curve}) if return_curve else (model, {"MSE":
mse})

def train_clf(depth, act, opt_name, return_curve=False):
    model = MLP(depth, HIDDEN, act)
    opt = make_opt(model.parameters(), opt_name)
    crit = nn.BCEWithLogitsLoss()
    dl = DataLoader(TensorDataset(torch.from_numpy(X2tr), torch.from_numpy(y2tr)),
                    batch_size=BATCH, shuffle=True)

    curve = []
    for _ in range(EPOCHS):
        for xb, yb in dl:

```

```

        opt.zero_grad(); loss = crit(model(xb), yb); loss.backward(); opt.step()
    if return_curve:
        with torch.no_grad():
            curve.append(crit(model(torch.from_numpy(X2tr)),
torch.from_numpy(y2tr)).item())
        with torch.no_grad():
            logits = model(torch.from_numpy(X2te))
            probs = torch.sigmoid(logits).numpy().ravel()
            acc = accuracy_score(y2te.ravel(), (probs>=0.5).astype(np.float32))
            auc = roc_auc_score(y2te.ravel(), probs)
        return (model, {"ACC": acc, "AUROC": auc, "curve": curve}) if return_curve else
(model, {"ACC": acc, "AUROC": auc})

# ----- Experiment Loop -----
rows = []
models_for_viz = {}
curves = {} # (task, depth, act, opt) -> list

for depth in DEPTHS:
    for act in ACTS:
        for opt in OPTS:
            m1, met1 = train_reg(depth, act, opt, return_curve=True)
            rows.append(dict(task="f1", depth=depth, activation=act, optimizer=opt,
**{k:v for k,v in met1.items() if k!="curve"}))
            curves[("f1",depth,act,opt)] = met1["curve"]
            key = ("f1", depth)
            if key not in models_for_viz or met1["MSE"] <
models_for_viz[key][1]["MSE"]:
                models_for_viz[key] = (m1, {"MSE": met1["MSE"]})

            m2, met2 = train_clf(depth, act, opt, return_curve=True)
            rows.append(dict(task="f2", depth=depth, activation=act, optimizer=opt,
**{k:v for k,v in met2.items() if k!="curve"}))
            curves[("f2",depth,act,opt)] = met2["curve"]
            key = ("f2", depth)
            if key not in models_for_viz or (met2["ACC"], met2["AUROC"]) >
(models_for_viz[key][1].get("ACC",0), models_for_viz[key][1].get("AUROC",0)):
                models_for_viz[key] = (m2, {"ACC": met2["ACC"], "AUROC":
met2["AUROC"]})

df = pd.DataFrame(rows)
df_f1 =
df[df.task=="f1"][['task','depth','activation','optimizer','MSE']].sort_values('MSE')
df_f2 =
df[df.task=="f2"][['task','depth','activation','optimizer','ACC','AUROC']].sort_values
(['ACC','AUROC'], ascending=False)
print(df_f1.to_string(index=False))
print(df_f2.to_string(index=False))

```

```

# ----- Visualizations -----

def plot_curve(task, depth, act):
    import matplotlib.pyplot as plt
    c_sgd = curves[(task, depth, act, "sgd")]
    c_mom = curves[(task, depth, act, "sgdm")]
    plt.figure()
    plt.plot(c_sgd, label="SGD")
    plt.plot(c_mom, label="SGD+momentum")
    plt.xlabel("Epoch"); plt.ylabel("Train loss"); plt.title(f"{task} • {depth} • {act}")
    plt.legend(); plt.show()

plot_curve("f1", "deep", "relu")
plot_curve("f2", "shallow", "relu")

def heatmaps_for_task(task_key, title_suffix):
    model, metrics = models_for_viz[task_key]
    gx, gy = np.meshgrid(np.linspace(0,1,200), np.linspace(0,1,200))
    grid = np.stack([gx.ravel(), gy.ravel()], axis=1).astype(np.float32)

    if task_key[0] == "f1":
        true = np.sin(2*np.pi*gx) * np.sin(2*np.pi*gy)
        with torch.no_grad():
            pred = model(torch.from_numpy(grid)).numpy().reshape(gx.shape)
        # true
        plt.figure(); plt.imshow(true, origin="lower", extent=[0,1,0,1],
aspect="equal"); plt.colorbar()
        plt.title(f"f1 true: sin(2πx) sin(2πy)")
        plt.xlabel("x"); plt.ylabel("y"); plt.show()
        # predicted
        plt.figure(); plt.imshow(pred, origin="lower", extent=[0,1,0,1],
aspect="equal"); plt.colorbar()
        plt.title(f"f1 predicted ({title_suffix}) • MSE={metrics['MSE']:.3e}")
        plt.xlabel("x"); plt.ylabel("y"); plt.show()
    else:
        true = (gx > gy).astype(float) # diagonal labeled 0
        with torch.no_grad():
            pred =
torch.sigmoid(model(torch.from_numpy(grid))).numpy().reshape(gx.shape)
        # true
        plt.figure(); plt.imshow(true, origin="lower", extent=[0,1,0,1],
aspect="equal"); plt.colorbar()
        plt.title(f"f2 true: 1[x>y] (x=y labeled 0)")
        plt.xlabel("x"); plt.ylabel("y"); plt.show()
        # predicted
        plt.figure(); plt.imshow(pred, origin="lower", extent=[0,1,0,1],
aspect="equal"); plt.colorbar()

```

```

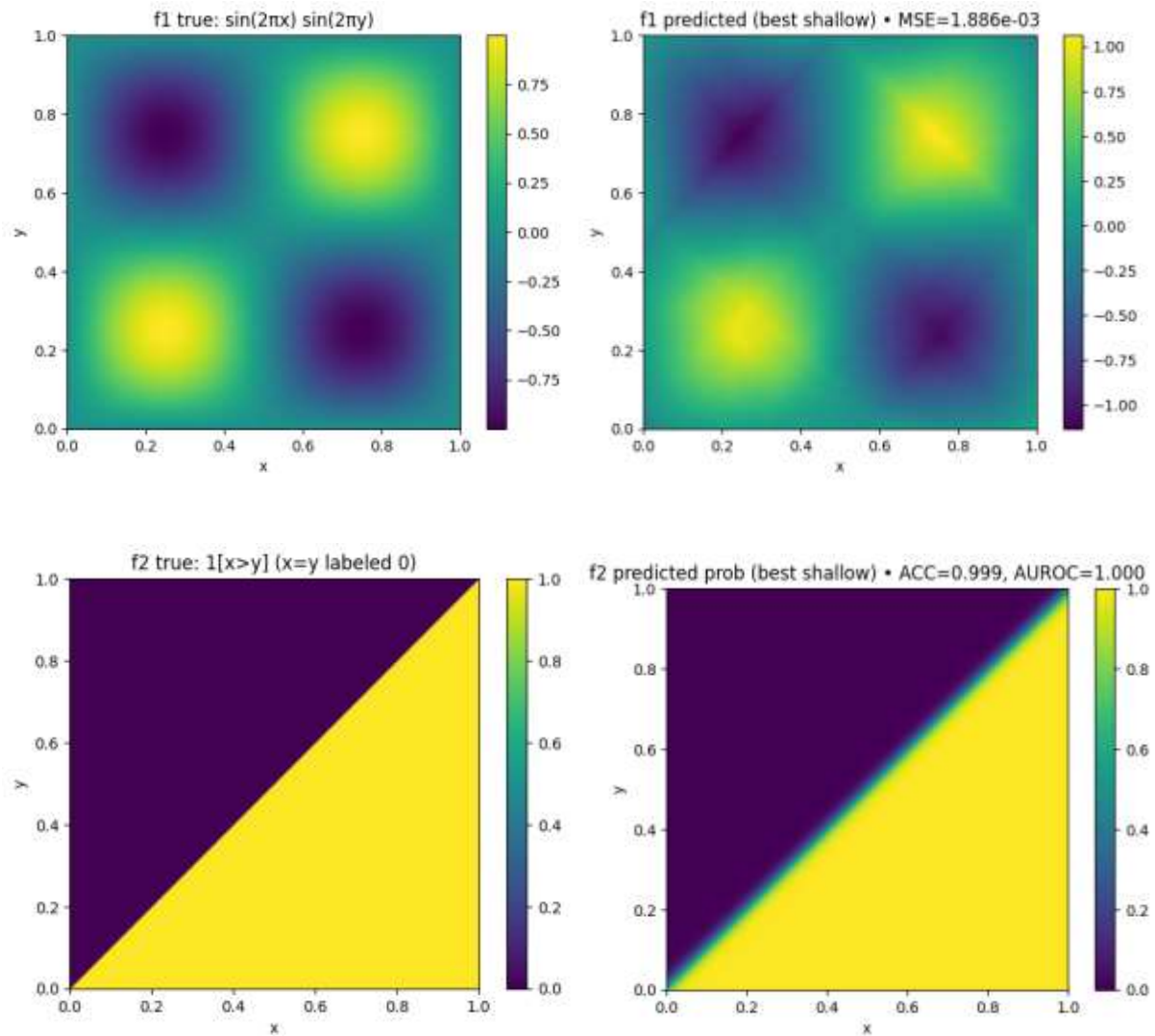
    acc = metrics['ACC']; auc = metrics['AUROC']
    plt.title(f"f2 predicted prob ({title_suffix}) • ACC={acc:.3f},
AUROC={auc:.3f}")
    plt.xlabel("x"); plt.ylabel("y"); plt.show()

# Best shallow models
heatmaps_for_task(("f1","shallow"), "best shallow")
heatmaps_for_task(("f2","shallow"), "best shallow")

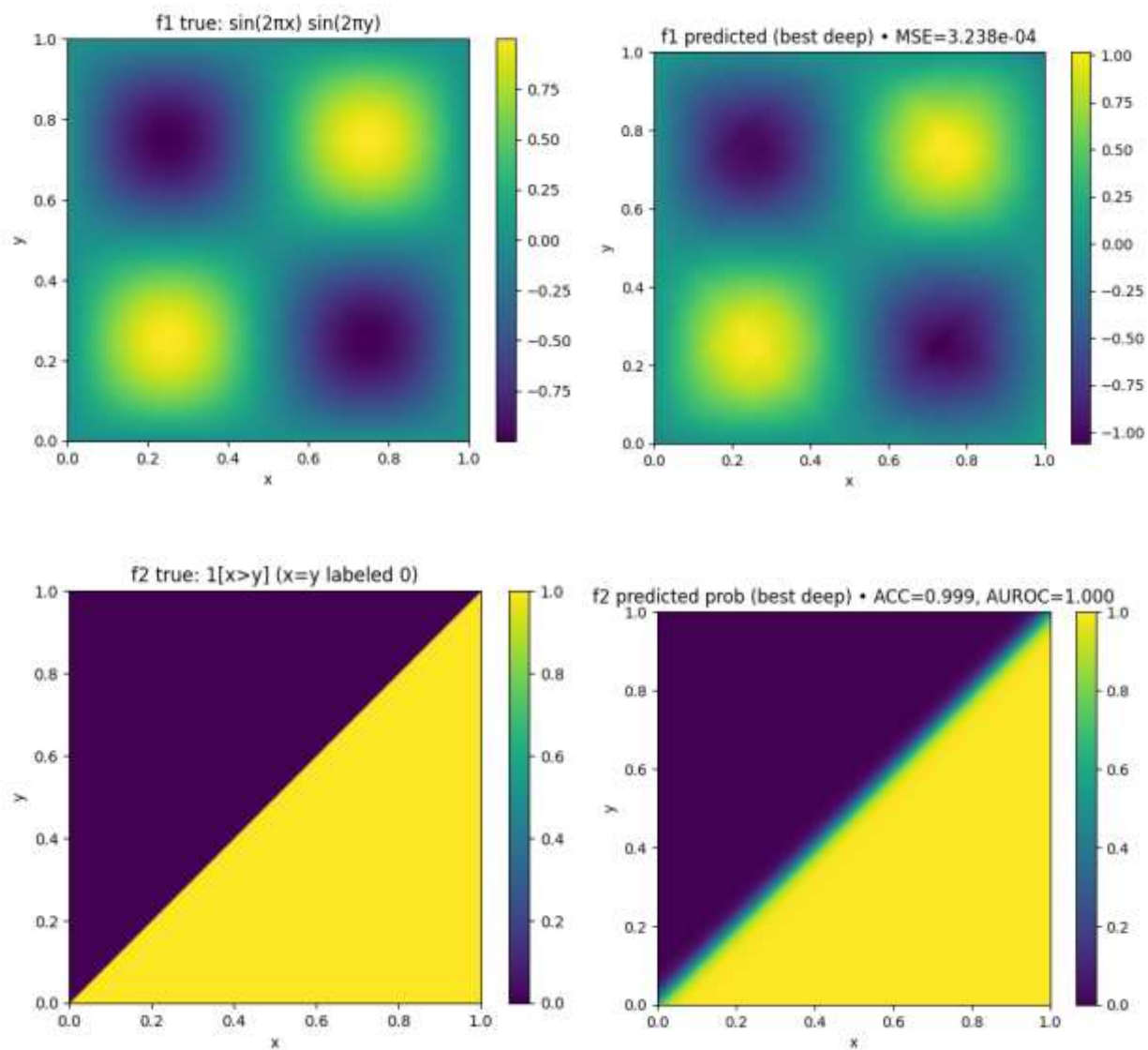
# Best deep models
heatmaps_for_task(("f1","deep"), "best deep")
heatmaps_for_task(("f2","deep"), "best deep")

```

Figures: True Vs Best Shallow for F1 and F2:



True Vs Best Deep Networks for F1 and F2:



9. References

- Leshno, M., Lin, V. Ya., Pinkus, A., & Schocken, S. (1993). Multilayer feedforward networks with a nonpolynomial activation function can approximate any function. *Neural Networks*, 6(6), 861–867.
- Cybenko, G. (1989). *Approximation by superpositions of a sigmoidal function*. *MCSS* 2:303–314.
- Hornik, K. (1989). *Multilayer feedforward networks are universal approximators*. *Neural Networks* 2(5):359–366.
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press, Chs. 6–7.
- Sitzmann, V. et al. (2020). *Implicit Neural Representations with Periodic Activation Functions (SIREN)*.

Libraries referenced:

- PyTorch, BCEWithLogitsLoss (numerically stable sigmoid + BCE). [PyTorch Docs](#), torch.optim.Adam. [PyTorch Docs](#), torch.optim.SGD (with momentum and SGD vanilla). [PyTorch Docs](#), Datasets & DataLoader tutorial (mini-batching, shuffling). [PyTorch Docs](#)
- scikit-learn roc_auc_score (AUROC). [scikit-learn.org](#), accuracy_score (accuracy). [scikit-learn.org](#)
- Matplotlib plt.imshow (2D scalar fields as heatmaps). [matplotlib.org](#)
- NumPy, numpy.random.rand (uniform sampling on [0,1)[0,1)[0,1)). [numpy.org](#)